

UAM Azcapotzalco, División de CBI

Club de Algoritmos

UAM Azcapotzalco

Entrenador: Dr. Rodrigo Alexander Castro Campos

1	Prólogo	1	Subsecuencia creciente más larga . . .	5	Cerco convexo	10			
			Trie	5	Distancia entre dos puntos	10			
2	Aritmética y primalidad	2	5	Combinatorios	5	8	Gráficas	10	
	Aritmética modular	2		Cadenas binarias usando recursión . .	5		Algoritmo de Bellman-Ford	10	
	Criba de Eratóstenes	2		Cadenas numéricas usando recursión .	5		Algoritmo de Edmonds-Karp	11	
	Euclides extendido	2		Coeficiente binomial iterativo	6		Algoritmo de Edmonds-Karp con cos-		
	Factorización básica	2		Coeficiente binomial modular	6		tos usando Bellman-Ford	11	
	Factorización de números grandes . . .	2		Código Lehmer	6		Algoritmo de Edmonds-Karp con cos-		
	Inverso modular	3		Permutaciones usando la biblioteca . .	6		tos usando el algoritmo de Johnson	12	
	Matriz como clase	3		Permutaciones usando recursión	6		Algoritmo de Floyd	12	
	Potencia rápida generalizada	3		6	Estructuras de datos	6	Algoritmo de Hopcroft-Karp	12	
	Potencia rápida modular	3			Ancastro común más bajo	6	Algoritmo de Kruskal	13	
	Primalidad Miller Rabin	3			Descomposición heavy-light	6	Algoritmo de Kuhn	13	
3	Búsqueda y ordenamiento	3			Preprocesamiento de árboles	7	Algoritmo de push-relabel	13	
	Búsqueda binaria generalizada	3			Árbol de Fenwick	7	Algoritmos de Dijkstra y Prim	14	
	Búsqueda ternaria generalizada	3			Árbol de Fenwick multidimensional . .	8	Circuito euleriano	14	
	Salto exponencial	4			Árbol de segmentos	8	Puentes de una gráfica	14	
4	Cadenas	4			Árbol de segmentos perezoso	9	Unión-pertenencia	14	
	Algoritmo Knuth-Morris-Pratt	4			Árbol de segmentos persistente	9			
	Arreglo de sufijos	4			Árboles con estadísticas	10	9	Lectura y escritura	14
	LCS con memoria lineal	4					Construcción de cadenas	14	
	LCS con programación dinámica . . .	4	7	Geométricos	10		Tokenización de líneas	15	

Prólogo (1)

Antes de enviar:

- Escribe algunos casos de prueba simples si los ejemplos no son suficientes.
- ¿Están cerca los límites de tiempo? Si es así, genera casos máximos.
- ¿El uso de memoria está dentro de los límites?
- ¿Podría haber algún desbordamiento?
- Asegúrate de enviar el archivo correcto.

Respuesta incorrecta:

- ¡Imprime tu solución! Imprime también la salida de depuración.
- ¿Estás limpiando todas las estructuras de datos entre casos de prueba?
- ¿Puede tu algoritmo manejar todo el rango de entrada?
- Lee nuevamente el enunciado completo del problema.
- ¿Manejas todos los casos límite correctamente?
- ¿Has entendido correctamente el problema?
- ¿Hay variables no inicializadas?
- ¿Algún desbordamiento?
- ¿Confundes N y M, i y j, etc.?
- ¿Estás seguro de que tu algoritmo funciona?
- ¿Qué casos especiales no has considerado?
- ¿Estás seguro de que las funciones STL que usas funcionan como piensas?
- Añade algunas aserciones, tal vez vuelve a enviar.
- Crea algunos casos de prueba para probar tu algoritmo.
- Revisa el algoritmo con un caso simple.
- Revisa esta lista nuevamente.
- Explica tu algoritmo a un compañero de equipo.
- Pídele a un compañero que revise tu código.
- Da un pequeño paseo, por ejemplo, al baño.

- ¿Es correcto tu formato de salida? (incluyendo espacios en blanco)
- Reescribe tu solución desde el principio o deja que un compañero lo haga.

Error de tiempo de ejecución:

- ¿Has probado todos los casos límite localmente?
- ¿Hay variables no inicializadas?
- ¿Estás leyendo o escribiendo fuera del rango de algún vector?
- ¿Alguna aserción que podría fallar?
- ¿Alguna posible división por 0? (por ejemplo, mod 0)
- ¿Alguna posible recursión infinita?
- ¿Punteros o iteradores invalidados?
- ¿Estás usando demasiada memoria?
- Depura con reenvíos (por ejemplo, señales remapeadas, ver Variables).

Límite de tiempo excedido:

- ¿Tienes algún bucle infinito posible?
- ¿Cuál es la complejidad de tu algoritmo?
- ¿Estás copiando muchos datos innecesarios? (usa referencias)
- ¿Qué tan grandes son la entrada y salida? (considera scanf)
- Evita vector, map. (usa arrays/unordered_map)
- ¿Qué piensan tus compañeros sobre tu algoritmo?

Límite de memoria excedido:

- ¿Cuál es la cantidad máxima de memoria que debería necesitar tu algoritmo?
- ¿Estás limpiando todas las estructuras de datos entre casos de prueba?

template.cpp

6 líneas

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    cin.tie(0)->sync_with_stdio(0);
}
```

setup.sh	2 líneas
# tambien se pueden usar comas: {a, x, m, l} touch {a..l}.in; tee {a..l}.cpp < template.cpp	
hash.sh	4 líneas
# Hashea un archivo, ignorando todos los espacios en blanco y # comentarios. Úsalo para verificar que el código fue escrito # correctamente. cpp -dD -P -fpreprocessed tr -d '[:space:]' md5sum cut -c-6	
.bashrc	2 líneas
alias c='g++ -Wall -Wconversion -Wfatal-errors -g -std=c++20 \ -fsanitize=undefined,address'	

Aritmética y primalidad (2)

Aritmética modular

Descripción: Operadores para aritmética modular. Supone que <i>MOD</i> es primo.	
Requiere: Inverso modular	79e848, 31 líneas

```
template<int MOD>
struct modular_int {
    int v;

    modular_int(int64_t u = 0)
    : v(u % MOD) {
        v += (v < 0) * MOD;
    }
    modular_int& operator+=(modular_int o) {
        if ((v += o.v) >= MOD) v -= MOD;
        return *this;
    }
    modular_int& operator-=(modular_int o) {
        if ((v -= o.v) < 0) v += MOD;
        return *this;
    }
    modular_int& operator*=(modular_int o) {
        v = (int64_t)v * o.v % MOD;
        return *this;
    }
    modular_int& operator/=(modular_int o) {
        return (*this) *= modular_int(inverso(o.v, MOD));
    }
    auto operator<=>(const modular_int& o) const = default;
    explicit operator int( ) const { return v; }
    friend modular_int operator+(modular_int a, modular_int b) {
        ↪ return a += b; }
    friend modular_int operator-(modular_int a, modular_int b) {
        ↪ return a -= b; }
    friend modular_int operator*(modular_int a, modular_int b) {
        ↪ return a *= b; }
    friend modular_int operator/(modular_int a, modular_int b) {
        ↪ return a /= b; }
};
//using gf = modular_int<1e9+7>;
```

Criba de Eratóstenes

Descripción: Criba de Eratóstenes para encontrar todos los números primos hasta un límite dado. Los números primos se almacenan en <i>primos</i> , y <i>factor</i> guarda el guarda el mayor factor primo de cada número.	
Complejidad: $\mathcal{O}(n \log \log n)$	f302d4, 21 líneas

```
struct criba {
    int tope;
    vector<bool> es_primo;
    vector<int> menor_factor;

    criba(int t)
    : tope(t), es_primo(t + 1, true), menor_factor(t + 1) {
        es_primo[0] = es_primo[1] = false;
        for (int i = 2; i <= tope; i++) {
            if (es_primo[i]) {
                menor_factor[i] = i;
                for (auto j = (size_t)i * i; j <= tope; j += i) {
                    es_primo[j] = false;
                    if (menor_factor[j] == 0) {
                        menor_factor[j] = i;
                    }
                }
            }
        }
    }
};
```

```
}
};
```

Euclides extendido

Descripción: Encuentra dos enteros <i>x</i> e <i>y</i> , tales que $ax + by = \gcd(a, b)$.	
Complejidad: $\mathcal{O}(\log(\min(a, b)))$	954285, 11 líneas

```
int64_t euclides_extendido(int64_t a, int64_t b, int64_t& x,
↪ int64_t& y) {
    x = 1, y = 0;
    int64_t x1 = 0, y1 = 1, a1 = a, b1 = b;
    while (b1) {
        int64_t q = a1 / b1;
        tie(x, x1) = make_tuple(x1, x - q * x1);
        tie(y, y1) = make_tuple(y1, y - q * y1);
        tie(a1, b1) = make_tuple(b1, a1 - q * b1);
    }
    return a1;
}
```

Factorización básica

Descripción: Devuelve un vector ordenado con los factores primos de <i>n</i> .	
Complejidad: $\mathcal{O}(\sqrt{n})$	6631b4, 14 líneas

```
vector<int64_t> factoriza(int64_t n) {
    vector<int64_t> factores;
    int64_t raiz = round(sqrt(n));
    for (int64_t x = 2; x <= raiz; ++x) {
        while (n % x == 0) {
            factores.push_back(x);
            n /= x;
        }
    }
    if (n > 1) {
        factores.push_back(n);
    }
    return factores;
}
```

Factorización de números grandes

Descripción: Algoritmo de Pollard Rho. <i>factoriza</i> toma un número entero $n \geq 2$ y devuelve un vector con los factores primos de <i>n</i> en orden arbitrario. Por ejemplo, para $n = 2299$, puede devolver {11, 19, 11}.	
Complejidad: $\mathcal{O}\left(n^{1/4}\right)$	
Requiere: Primalidad Miller Rabin	e7b23c, 35 líneas

```
int64_t factor_pollard_rho(int64_t n, int64_t c) {
    int64_t x = 2, y = 2, k = 2;
    for (int64_t i = 2; ; ++i) {
        x = ((__int128_t(x) * x) % n) + c;
        if (x >= n) {
            x -= n;
        }
        int64_t d = gcd(x - y, n);
        if (d != 1) {
            return d;
        }
        if (i == k) {
            y = x, k *= 2;
        }
    }
}

vector<int64_t> factoriza(int64_t n) { // suposición: n >= 2
    vector<int64_t> factores;
    if (es_primo(n)) {
        factores.push_back(n);
    } else {
        for (int64_t i = 2; ; i++) {
            auto checar = factor_pollard_rho(n, i);
            if (checar != n) {
                vector<int64_t> factores1 = factoriza(checar);
                vector<int64_t> factores2 = factoriza(n / checar);
                factores.insert(factores.end( ), factores1.begin( ),
                    ↪ factores1.end( ));
                factores.insert(factores.end( ), factores2.begin( ),
                    ↪ factores2.end( ));
                break;
            }
        }
    }
    return factores;
}
```

Inverso modular

Descripción: Encuentra x en $[0,m)$ tal que $ax \equiv 1 \bmod m$ (Usa *inverso_compuesto* si m no es primo).

Complejidad: $\mathcal{O}(\log(m))$

Requiere: Euclides extendido 4d0714, 13 líneas

```
constexpr int64_t inverso(int64_t a, int64_t m) {
    return a <= 1 ? a : m - (m/a) * inverso(m % a, m) % m;
}
```

```
int64_t inverso_compuesto(int64_t a, int64_t m) {
    int64_t x, y;
    int64_t g = euclides_extendido(a, m, x, y);
    x %= m;
    if (x < 0) {
        x += m;
    }
    return x;
}
```

Matriz como clase

Descripción: Operaciones básicas con matrices.

Uso: matriz<2, 2>({{ { 0, 1 }, { 1, 1 } }}

Complejidad: $\mathcal{O}(n^3)$ 4dc7ac, 29 líneas

```
template<class T, int F, int C>
struct matriz : array<array<T, C>, F> {
    explicit matriz(bool identidad = false) {
        for (int i = 0; i < F; ++i) {
            for (int j = 0; j < C; ++j) {
                (*this)[i][j] = (i == j && identidad);
            }
        }
    }
    explicit matriz(const array<array<T, C>, F>& m)
    : array<array<T, C>, F>(m) {}
}
```

```
template<int D>
matriz<T, F, D> operator*(const matriz<T, C, D>& m) {
    matriz<T, F, D> res;
    for (int i = 0; i < F; ++i) {
        for (int j = 0; j < D; ++j) {
            for (int k = 0; k < C; ++k) {
                res[i][j] += (*this)[i][k] * m[k][j];
            }
        }
    }
    return res;
}
void operator*=(const matriz<T, F, C>& m) {
    *this = *this * m;
}
};
```

Potencia rápida generalizada

Descripción: Calcula a^b para cualquier a que pertenezca a un monoide.

Complejidad: $\mathcal{O}(\log b)$. b2ed1d, 10 líneas

```
template<typename T>
T potencia(T a, int b) {
    T res(1);
    for (; b != 0; b /= 2, a *= a) {
        if (b % 2 == 1) {
            res *= a;
        }
    }
    return res;
}
```

Potencia rápida modular

Descripción: Calcula $a^b \bmod c$.

Complejidad: $\mathcal{O}(\log b)$ 402fbc, 11 líneas

```
int64_t potencia(int64_t b, int64_t e, int64_t mod) {
    int64_t res = 1; b %= mod;
    while (e != 0) {
        if (e % 2 == 1) {
            res = (__int128_t(res) * b) % mod;
        }
        b = (__int128_t(b) * b) % mod;
    }
```

```
    e /= 2;
}
return res;
}
```

Primalidad Miller Rabin

Descripción: Prueba de primalidad determinista de Miller-Rabin.

Complejidad: $\mathcal{O}(\log p)$.

Requiere: Potencia rápida modular fff63e, 32 líneas

```
bool es_primo(int64_t n) {
    if (n < 2) {
        return false;
    }

    int64_t d = n - 1, s = 0;
    while (d % 2 == 0) {
        d /= 2, s += 1;
    }
    auto compuesto_con = [&](int64_t a) {
        int64_t x = potencia(a, d, n);
        if (x == 1 || x == n - 1) {
            return false;
        }
        for (int64_t r = 1; r < s; ++r) {
            x = (__int128_t(x) * x) % n;
            if (x == n - 1) {
                return false;
            }
        }
        return true;
    };

    for (int64_t a : { 2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37 }) {
        if (n == a) {
            return true;
        } else if (compuesto_con(a)) {
            return false;
        }
    }
    return true;
}
```

Búsqueda y ordenamiento (3)

Búsqueda binaria generalizada

Descripción: Devuelve el primer x tal que $pred(x)$ es verdadero, y para todo x , $pred(x)$ debe ser una función monótona.

Uso: auto elemento = busqueda_binaria(0, 1000, [](auto valor) {
 return valor / 500 != 0;
});

Complejidad: $\mathcal{O}(\log(n))$ 9ba0ca, 12 líneas

```
template<typename T, typename F>
T busqueda_binaria(T ini, T fin, F pred) {
    while (ini != fin) {
        auto mitad = midpoint(ini, fin);
        if (pred(mitad)) {
            fin = mitad;
        } else {
            ini = mitad + 1;
        }
    }
    return fin;
}
```

Búsqueda ternaria generalizada

Descripción: Devuelve el primer x tal que $f(x)$ es el máximo de f, sobre una función f unimodal.

Uso: auto f = [](double x) { return -pow((x - 3), 2) + 9; };
double i = busqueda_ternaria(-100.0, +100.0, f, 1e-6);
// se puede usar un rango entero y omitir la tolerancia

Complejidad: $\mathcal{O}(\log(n))$ b07c53, 19 líneas

```
template<typename T, typename F>
T busqueda_ternaria(T ini, T fin, F funcion, T tol = 1) {
    while (fin - ini >= 3 * tol) {
        T mitad1 = ini + 1 * (fin - ini) / 3;
        T mitad2 = ini + 2 * (fin - ini) / 3;
        auto v1 = funcion(mitad1), v2 = funcion(mitad2);
        if (v1 <= v2) {
            ini = mitad1 + tol;
        }
        if (v1 >= v2) {
            fin = mitad2;
        }
    }
```

```

    }
}
pair mejor = { ini, funcion(ini) };
for (T i = ini + tol; i < fin; i += tol) {
    mejor = max(mejor, { i, funcion(i) });
}
return mejor.first;
}

```

Salto exponencial

Descripción: Similar a la búsqueda binaria, pero ofrece un mejor rendimiento asintótico cuando el elemento buscado está cerca del inicio de la lista.

Complejidad: $\mathcal{O}(\log(i))$, donde i es la posición del elemento buscado o donde debería estar.

Requiere: Búsqueda binaria generalizada

e9b66e, 9 líneas

```

template<typename T, typename F>
T salto_exponencial(T ini, T fin, F pred) {
    auto probar = ini;
    while (probar != fin && !pred(probar)) {
        probar += min(fin - probar, probar - ini + 1);
    }

    return (probar == fin ? fin : busqueda_binaria(probar - (probar -
    ↪ ini) / 2, probar, pred));
}

```

Cadenas (4)

Algoritmo Knuth-Morris-Pratt

Descripción: La función `tabla_kmp[i]` calcula la longitud del prefijo más largo de la cadena s que termina en i , excluyendo la subcadena $s[0...i]$ en sí misma (por ejemplo, para $abacaba \rightarrow 0010123$). Puede usarse para encontrar todas las ocurrencias de una cadena.

Uso: `auto tabla = tabla_kmp(patron);`
`auto ocurrencias = busca(patron, texto, tabla);`

Complejidad: $\mathcal{O}(n)$

536d5b, 24 líneas

```

vector<size_t> tabla_kmp(const string& s) {
    vector<size_t> b = { size_t(-1) };
    for (size_t i = 0, j = -1; i < s.size(); ++i) {
        while (j != -1 && s[i] != s[j]) {
            j = b[j];
        }
        b.push_back(++j);
    }
    return b;
}

```

```

vector<size_t> busca(const string& s, const string& t, const
↪ vector<size_t>& b) {
    vector<size_t> res;
    for (size_t i = 0, j = 0; i < t.size(); ++i) {
        while (j != -1 && t[i] != s[j]) {
            j = b[j];
        }
        if (++j == s.size()) {
            res.push_back(i + 1 - s.size());
            j = b[j];
        }
    }
    return res;
}

```

Arreglo de sufijos

Descripción: Construye el arreglo de sufijos para una cadena. `suffix[i]` es el iterador del inicio del sufijo que ocupa la posición i en el arreglo de sufijos ordenado. La función `longest_prefix` calcula los prefijos comunes más largos para las cadenas vecinas en el arreglo de sufijos: $lcp[i] = lcp(suffix[i], suffix[i+1])$, y $lcp[n-1] = 0$. La función `substring_search` devuelve una pareja de iteradores sobre `suffix` que denotan el inicio y el fin de todos los sufijos donde la subcadena es prefijo.

Uso: `suffix_array sa(s.begin(), s.end());`
`auto [ini, fin] = sa.substring_search(b.begin(), b.end());`

Complejidad: $\mathcal{O}(n \log^2(n))$ donde n es la longitud de la cadena, y $\mathcal{O}(k \log(n))$ para `substring_search` donde k es la longitud del patrón.

7b6de5, 59 líneas

```

template<typename RI>
struct suffix_array {
    RI si, sf;
    vector<int> rank;
    vector<RI> suffix;

    suffix_array(RI ri, RI rf)
    : si(ri), sf(rf), rank(si, sf), suffix(sf - si) {

```

```

        vector<int> indices(sf - si);
        iota(indices.begin(), indices.end(), 0);
        for (int t = 1; t <= sf - si; t *= 2) { // importante que se
            ↪ haga para sf - si == 1 pues se debe normalizar el rank
            ↪ inicial
            auto pred = [&, rank = this->rank](int i1, int i2) {
                return make_pair(rank[i1], (i1 + t < sf - si ? rank[i1 + t]
                ↪ : -1)) < make_pair(rank[i2], (i2 + t < sf - si ? rank[i2]
                ↪ + t) : -1));
            };
            sort(indices.begin(), indices.end(), cref(pred));
            for (int i = 0, r = 0; i < indices.size(); ++i) {
                rank[indices[i]] = r;
                r += (i + 1 != indices.size() && pred(indices[i], indices[i]
                ↪ + 1));
            }
        }

        for (int i = 0; i < suffix.size(); ++i) {
            suffix[rank[i]] = si + i;
        }
    }

    vector<int> longest_prefix() {
        vector<int> res(sf - si);
        for (int i = 0, t = 0; i < rank.size(); ++i) {
            if (rank[i] + 1 != sf - si) {
                t += mismatch(si + i + t, sf, suffix[rank[i] + 1] + t,
                ↪ sf).first - (si + i + t);
                res[rank[i]] = t;
                t -= (t > 0);
            } else {
                t = 0;
            }
        }
        return res;
    }
}

```

```

auto substring_search(auto bi, auto bf) {
    struct comparador {
        const int pos;
        bool operator()(RI iter, char c) {
            return iter[pos] < c;
        }
        bool operator()(char c, RI iter) {
            return c < iter[pos];
        }
    };

    auto xi = suffix.begin(), xf = suffix.end();
    for (int i = 0; i < bf - bi; ++i) {
        auto temp = equal_range(xi, xf, bi[i], comparador{i});
        xi = temp.first, xf = temp.second;
    }
    return pair(xi, xf);
}
};

```

LCS con memoria lineal

Descripción: Regresa la longitud de la subsecuencia común más larga usando memoria lineal.

Complejidad: $\mathcal{O}(nm)$

fae303, 16 líneas

```

int lcs(const string& a, const string& b) {
    int mem[2][b.size() + 1];
    int *actual = mem[0], *previo = mem[1];
    for (int i = a.size(); i >= 0; --i, swap(actual, previo)) {
        for (int j = b.size(); j >= 0; --j) {
            if (i == a.size() || j == b.size()) {
                actual[j] = 0;
            } else if (a[i] == b[j]) {
                actual[j] = 1 + previo[j + 1];
            } else {
                actual[j] = max(actual[j + 1], previo[j]);
            }
        }
    }
    return previo[0];
}

```

LCS con programación dinámica

Descripción: Devuelve un vector de pares de enteros (i, j) que indican, respectivamente, las posiciones de las coincidencias en a y b .

Complejidad: $\mathcal{O}(n \cdot m)$

34cdec, 27 líneas

```

auto lcs(const string& a, const string& b) {
    int mem[a.size() + 1][b.size() + 1];
    for (int i = a.size(); i >= 0; --i) {
        for (int j = b.size(); j >= 0; --j) {
            if (i == a.size() || j == b.size()) {

```

```

        mem[i][j] = 0;
    } else if (a[i] == b[j]) {
        mem[i][j] = 1 + mem[i + 1][j + 1];
    } else {
        mem[i][j] = max(mem[i][j + 1], mem[i + 1][j]);
    }
}
}

vector<pair<int, int>> res;
int i = 0, j = 0;
while (i < a.size( ) && j < b.size( )) {
    if (a[i] == b[j]) {
        res.emplace_back(i++, j++);
    } else if (mem[i][j] == mem[i][j + 1]) {
        ++j;
    } else {
        ++i;
    }
}
return res;
}

```

Subsecuencia creciente más larga

Descripción: La función *lis_size* devuelve el tamaño de la subsecuencia creciente más larga, mientras que la función *lis* devuelve un vector de iteradores que componen dicha subsecuencia.

Uso: `vector<int> arr = { 10, 22, 9, 33, 21, 50, 41, 60 };
 auto tam = lis_size(arr.begin(), arr.end());`

Complejidad: $\mathcal{O}(n \log n)$

1532e5, 41 líneas

```

template<typename FI>
size_t lis_size(FI ini, FI fin) {
    vector<typename iterator_traits<FI>::value_type> valores;
    for (auto i = ini; i != fin; ++i) {
        auto cambiar = upper_bound(valores.begin( ), valores.end( ),
            *i); // upper_bound para creciente no estricta,
            // lower_bound para creciente estricta
        if (cambiar == valores.end( )) {
            valores.push_back(*i);
        } else {
            *cambiar = *i;
        }
    }

    return valores.size( );
}

template<typename BI>
vector<BI> lis(BI ini, BI fin) {
    vector<BI> posiciones(1), atras;
    auto pred = [&](BI i, BI j) {
        return *i < *j;
    };

    for (auto i = ini; i != fin; ++i) {
        auto cambiar = upper_bound(posiciones.begin( ) + 1,
            posiciones.end( ), i, pred); // lower_bound para creciente
            // estricta
        if (cambiar == posiciones.end( )) {
            atras.push_back(posiciones.back( ));
            posiciones.push_back(i);
        } else if (pred(i, *cambiar)) {
            // tautología con
            // upper_bound (creciente no estricta) pero no con lower_bound
            // (creciente estricta)
            atras.push_back(*(cambiar - 1));
            *cambiar = i;
        } else {
            atras.emplace_back( );
        }
    }

    for (auto i = posiciones.end( ); i != posiciones.begin( ) + 1;
        --i) {
        *(i - 2) = atras[*i - 1] - inil;
    }

    return vector<BI>(posiciones.begin( ) + 1, posiciones.end( ));
}

```

Trie

Descripción: Árbol enraizado que mantiene un conjunto de cadenas. La función *inserta* añade una cadena al trie, *posicion* devuelve el nodo donde termina una cadena (o *nullptr* si no existe), *busca* verifica si una cadena completa está presente, y *prefijo* comprueba si un prefijo dado existe en el trie.

Complejidad: $\mathcal{O}(n)$

b7cb46, 39 líneas

```

class trie {
public:

```

```

    bool inserta(const string& s) {
        auto actual = this;
        for (int i = 0; i < s.size( ); ++i) {
            auto& siguiente = actual->nivel_[s[i]];
            if (siguiente == nullptr) {
                siguiente = new trie;
            }
            actual = siguiente;
        }
        return actual->nivel_.emplace('\0', nullptr).second;
    }

    trie* posicion(const string& s) {
        auto actual = this;
        for (int i = 0; i < s.size( ); ++i) {
            auto iter = actual->nivel_.find(s[i]);
            if (iter == actual->nivel_.end( )) {
                return nullptr;
            }
            actual = iter->second;
        }
        return actual;
    }

    bool busca(const string& s) {
        auto pos = posicion(s);
        return pos != nullptr && pos->nivel_.find('\0') !=
            pos->nivel_.end( );
    }

    bool prefijo(const string& s) {
        auto pos = posicion(s);
        return pos != nullptr;
    }
}

```

```

private:
    map<char, trie*> nivel_;
};

```

Combinatorios (5)

Cadenas binarias usando recursión

Descripción: Generación de cadenas binarias usando recursión.

Complejidad: $\mathcal{O}(2^n \cdot n)$

4c5d67, 17 líneas

```

int n;
bool arr[MAX];

// llamada inicial: cadenas_binarias(0)
void cadenas_binarias(int i) {
    if (i == n) {
        for (int i = 0; i < n; ++i) {
            cout << arr[i];
        }
        cout << "\n";
    } else {
        arr[i] = false;
        cadenas_binarias(i + 1);
        arr[i] = true;
        cadenas_binarias(i + 1);
    }
}

```

Cadenas numéricas usando recursión

Descripción: Generación de cadenas numéricas usando recursión.

Complejidad: $\mathcal{O}(10^n \cdot n)$

e6c454, 17 líneas

```

int n;
int arr[MAX];

// llamada inicial: cadenas_numericas(0)
void cadenas_numericas(int i) {
    if (i == n) {
        for (int i = 0; i < n; ++i) {
            cout << arr[i] << " ";
        }
        cout << "\n";
    } else {
        for (int d = 0; d <= 9; ++d) {
            arr[i] = d;
            cadenas_numericas(i + 1);
        }
    }
}

```


Coeficiente binomial iterativo

Descripción: El número de subconjuntos de k elementos de un conjunto de n elementos, $\binom{n}{k} = \frac{n!}{k!(n-k)!}$.

Complejidad: $\mathcal{O}(k)$ 608c5f, 9 líneas

```
uint64_t binomial(int n, int k) {
    uint128_t res = 1;
    // uint64_t es más rápido pero falla en casos raros
    for (int i = 1; i <= k; ++i) {
        res *= n + 1 - i;
        res /= i;
    }
    return res;
}
```

Coeficiente binomial modular

Descripción: Calcula $\binom{n}{k} \bmod m$.

Complejidad: $\mathcal{O}(\log(m))$

Requiere: Inverso modular e61de7, 5 líneas

```
int64_t factorial[N + 1]; // N dado por el problema

int64_t binomial(int n, int k, int64_t m) {
    return factorial[n] * inverso(factorial[k] * factorial[n - k] % m,
        ↪ m) % m;
}
```

Código Lehmer

Descripción: Permutaciones a/desde enteros. La biyección preserva el orden lexicográfico.

Complejidad: $\mathcal{O}(n^2)$ 64d73f, 29 líneas

```
constexpr size_t factorial(size_t n) {
    return (n == 0 ? 1 : n * factorial(n - 1));
}

template<typename T>
size_t indice(T ini, T fin) {
    size_t n = fin - ini, r = factorial(n - 1), res = 0;
    for (T i = ini; i != fin; ++i) {
        res += r * count_if(i + 1, fin, [&](size_t v) { return v < *i;
            ↪ });
        if (fin - i - 1 != 0) {
            r /= fin - i - 1;
        }
    }
    return res;
}

template<typename T>
void permutacion(T ini, T fin, size_t indice) {
    iota(ini, fin, size_t(0));
    size_t n = fin - ini, r = factorial(n - 1);
    for (T i = ini; i != fin; ++i) {
        size_t d = indice / r; indice %= r;
        rotate(i, i + d, i + d + 1);
        if (fin - i - 1 != 0) {
            r /= fin - i - 1;
        }
    }
}
```

Permutaciones usando la biblioteca

Descripción: Generación de permutaciones usando la biblioteca.

Complejidad: $\mathcal{O}(n! \cdot n)$ ea4a9f, 5 líneas

```
int n;
int arr[MAX]; // inicializar con { 0, 1, ..., n - 1 }
do {
    // procesar
} while (next_permutation(&arr[0], &arr[n]));
```

Permutaciones usando recursión

Descripción: Generación de permutaciones usando recursión.

Complejidad: $\mathcal{O}(n! \cdot n)$ ee428a, 18 líneas

```
int n;
int arr[MAX]; // inicializar con { 0, 1, ..., n - 1 }
```

```
// llamada inicial: permutaciones(0)
void permutaciones(int i) {
    if (i == n) {
        for (int i = 0; i < n; ++i) {
            cout << arr[i] << " ";
        }
        cout << "\n";
    } else {
        for (int j = i; j < n; ++j) {
            swap(arr[i], arr[j]);
            permutaciones(i + 1);
            swap(arr[i], arr[j]);
        }
    }
}
```

Estructuras de datos (6)

Ancestro común más bajo

Descripción: Calcula el ancestro común de dos vértices a partir de haber precalculado un preorden aumentado o contorno.

Uso: `lowest_common_ancestor lca(tree_stats(raiz, adyacencia));`
`int ancestro = lca.query(i, j);`

Complejidad: $\mathcal{O}(n \log n)$ para el preprocesamiento y $\mathcal{O}(1)$ para las queries.

Requiere: Preprocesamiento de árboles 673d14, 29 líneas

```
struct lowest_common_ancestor {
    lowest_common_ancestor(tree_stats&& s)
    : primera(s.alturas.size( )), alturas(move(s.alturas)) {
        int tam = tabla.emplace_back(move(s.contorno)).size( );
        for (int i = tam - 1; i >= 0; --i) {
            primera[tabla[0][i]] = i;
        }
        for (int k = 2; k <= tam; k *= 2) {
            vector<int>& fila = tabla.emplace_back(tabla.back( ));
            for (int i = 0; i < tam; ++i) {
                fila[i] = selector(fila[i], fila[min(i + k / 2, tam - 1)]);
            }
        }
    }

    int query(int i, int j) const {
        int ini = min(primera[i], primera[j]), fin = max(primera[i],
            ↪ primera[j]) + 1;
        int t = bit_floor(unsigned(fin - ini)), x =
            ↪ countr_zero(unsigned(t));
        return selector(tabla[x][ini], tabla[x][ini + (fin - ini - t)]);
    }

    int selector(int i, int j) const {
        return (alturas[i] > alturas[j] ? i : j);
    }

private:
    vector<int> primera, alturas;
    vector<vector<int>>> tabla;
};
```

Descomposición heavy-light

Descripción: Descomposición de un árbol con ejemplo de queries en el camino entre dos vértices

Uso: `auto hv = make_hld_vertex(raiz, move(adyacencia),`
`move(vector_costos), assoc_op o lazy_assoc_op);`
`hv.replace(i, 7);`
`int r1 = hv.query(i, j);`
`auto he = make_hld_edge(raiz, move(vector_inicial),`
`move(mapa_costos), assoc_op o lazy_assoc_op);`
`he.update_with(i, j, 7); // requiere lazy`
`int r2 = he.query(i, j);`

Complejidad: $\mathcal{O}(n \log n)$ para procesar, $\mathcal{O}(\log^2 n)$ por query

Requiere: Preprocesamiento de árboles abdae0, 113 líneas

```
template<typename OP, typename F, bool VMODE>
struct hld_base {
    using T = decltype(OP::neutro);

    hld_base(int raiz, vector<vector<int>>&& adj, OP&& op, F&& fc,
        ↪ bool constant<VMODE>)
    : subarbol(adj.size( )), invertido(adj.size( )) {
        tree_stats stats(0, adj);
        alturas = move(stats.alturas), pesos = move(stats.pesos);
        vector<int> grupo = { raiz }; vector<T> costos;
        descomposicion(raiz, -1, adj, grupo, costos, fc);
        st.emplace(move(costos), move(op));
    }
```

```

auto& segment_tree( ) const {           // sólo se se usará visit
    ↪ explícitamente
    return *st;
}

int lowest_common_ancestor(int i, int j) const { // con hld se
    ↪ puede implementar el lca de otra forma
    while (invertido[i].first != invertido[j].first) {
        int ri = grupos[invertido[i].first].first, rj =
            ↪ grupos[invertido[j].first].first;
        (pair(alturas[ri], alturas[i]) < pair(alturas[rj], alturas[j]))
            ↪ ? i = ri : j = rj);
    }
    return (alturas[i] >= alturas[j] ? i : j);
}

void replace(int i, const T& c) {         // sólo si se usa
    ↪ hld vertex y segment tree
    static_assert(VMODE);
    st->replace(invertido[i].second, c);
}

T subtree_query(int i) const {
    return st->query(invertido[i].second, invertido[i].second +
        ↪ pesos[i] - !VMODE);
}

void subtree_update_with(int i, auto u) { // sólo si se usa
    ↪ lazy segment tree
    return st->update_with(invertido[i].second, invertido[i].second
        ↪ + pesos[i] - !VMODE, u);
}

template<typename V>
void subtree_visit(int i, V&& vis) {
    return st->visit(invertido[i].second, invertido[i].second +
        ↪ pesos[i] - !VMODE, vis);
}

T path_query(int i, int j) const {
    T res = st->op.neutro;
    path_visit(i, j, [&](int v, int ini, int fin) {
        res = st->op.funcion(res, st->query(ini, fin));
    });
    return res;
}

void path_update_with(int i, int j, auto u) { // sólo si se usa
    ↪ lazy segment tree
    path_visit(i, j, [&](int v, int ini, int fin) {
        st->update_with(ini, fin, u);
    });
}

template<typename V>
void path_visit(int i, int j, V&& vis) const {
    int lca = lowest_common_ancestor(i, j);
    vis(lca, invertido[lca].second, invertido[lca].second + VMODE);
    for (int hijo : {i, j}) {
        while (hijo != lca) {
            auto [subir, pos_tope] = (invertido[hijo].first ==
                ↪ invertido[lca].first ? pair(lca, invertido[lca].second +
                ↪ VMODE) : grupos[invertido[hijo].first]);
            vis(hijo, pos_tope, invertido[hijo].second + VMODE);
            hijo = subir;
        }
    }
}

private:
void descomposicion(int actual, int anterior, vector<vector<int>>&
    ↪ adj, vector<int>& grupo, vector<T>& costos, F& fc) {
    erase(adj[actual], anterior);
    sort(adj[actual].begin( ), adj[actual].end( ), [&](int i, int j)
        ↪ {
        return pesos[i] < pesos[j];
    });

    if (!adj[actual].empty( ) && 2 * pesos[adj[actual].back( )] >=
        ↪ pesos[actual]) {
        grupo.push_back(adj[actual].back( ));
        descomposicion(adj[actual].back( ), actual, adj, grupo,
            ↪ costos, fc);
        adj[actual].pop_back( );
    } else {
        grupos.emplace_back(grupo[0], costos.size( ));
        for (int i = (!VMODE || costos.size( ) != 0); i < grupo.size(
            ↪ ); ++i) {
            invertido[grupo[i]] = { grupos.size( ) - 1, costos.size( ) +
                ↪ !VMODE };
            costos.push_back(fc(grupo[i - 1], grupo[i]));
        }
    }
    for (int v : adj[actual]) {
        grupo = { actual, v };
        descomposicion(v, actual, adj, grupo, costos, fc);
    }
}

```

```

    }
}

vector<int> alturas, pesos;
vector<pair<int, int>> subarbol;
vector<pair<int, int>> invertido, grupos; // (id_grupo, pos) y
    ↪ (repr, pos_ini)
optional<decltype>(make_segment_tree(vector<T>( ), declval<OP>(
    ↪ ))> st;

template<typename OP, typename T>
auto make_hld_vertex(int raiz, vector<vector<int>>&& adj,
    ↪ vector<T>&& costos, OP&& op) {
    return hld_base(raiz, move(adj), move(op), [c = move(costos)](int
        ↪ i, int j) {
        return c[j];
    }, bool_constant<true>( ));
}

template<typename OP, typename T>
auto make_hld_edge(int raiz, vector<vector<int>>&& adj,
    ↪ map<pair<int, int>, T>&& costos, OP&& op) {
    return hld_base(raiz, move(adj), move(op), [c = move(costos)](int
        ↪ i, int j) {
        return c.find({ i, j })->second;
    }, bool_constant<false>( ));
}

```

Preprocesamiento de árboles

Descripción: DFS estándar que preprocesa un árbol para calcular algunos datos útiles.

Uso: tree_stats stats(raiz, adyacencia);

Complejidad: $\mathcal{O}(n)$.

d91f57, 21 líneas

```

struct tree_stats {
    vector<int> alturas, pesos, contorno; // alturas siempre, pesos
    ↪ para hld, contorno para lca
    tree_stats(int raiz, const vector<vector<int>>&& adj)
        : alturas(adj.size( )), pesos(adj.size( )) {
        calcula(raiz, -1, adj);
    }

private:
void calcula(int actual, int anterior, const vector<vector<int>>&
    ↪ adj) {
    int altura = 0, peso = 0;
    contorno.push_back(actual);
    for (auto v : adj[actual]) {
        if (v != anterior) {
            calcula(v, actual, adj);
            altura = max(altura, alturas[v]), peso += pesos[v];
            contorno.push_back(actual);
        }
    }
    alturas[actual] = altura + 1, pesos[actual] = peso + 1;
}
}

```

Árbol de Fenwick

Descripción: Calcula las sumas parciales $a[i] + a[i+1] + \dots + a[f-1]$, y actualiza o reemplaza elementos individuales $a[i]$.

Complejidad: $\mathcal{O}(\log n)$

9baa26, 51 líneas

```

template<typename T>
class fenwick_tree {
public:
    fenwick_tree(int n)
        : mem_(n + 1) {
    }

    int size( ) const {
        return mem_.size( ) - 1;
    }

    T operator[](int i) const {
        return query(i, i + 1);
    }

    T query(int i, int f) const {
        return query_until(f) - query_until(i);
    }

    T query_until(int f) const {
        T res = 0;
        for (; f != 0; f -= (f & -f)) {
            res += mem_[f];
        }
        return res;
    }
}

```



```

}

int min_prefix(T v) const {
    ↪ // calcula la cantidad mínima
    ↪ de elementos (comenzando por la izquierda) que se necesitan
    ↪ para lograr un acumulado >= v;
    int i = 0; // si es imposible lograr dicha
    ↪ suma, regresa .size( ) + 1
    for (int j = bit_floor(mem_.size( )); j > 0; j /= 2) {
        if (i + j < mem_.size( ) && mem_[i + j] < v) {
            v -= mem_[i + j];
            i += j;
        }
    }
    return i + (v > 0);
}

void replace(int i, const T& v) {
    modify_add(i, v - operator[](i));
}

void modify_add(int i, const T& d) {
    for (i += 1; i < mem_.size( ); i += (i & -i)) {
        mem_[i] += d;
    }
}

private:
vector<T> mem_;
};

```

Árbol de Fenwick multidimensional

Descripción: Realiza actualizaciones y consultas de rango en múltiples dimensiones.

Uso: fenwick_tree<int, 2> arbol_2d(3, 3);
 int suma = arbol_2d.query(1, 1, 3, 3); // $\sum_{i=1}^2 \sum_{j=1}^2 a[i][j]$

Complejidad: $\mathcal{O}(4^d \log n_1 \cdot \log n_2 \cdots \log n_d)$. Es eficiente para dimensiones bajas (hasta 4 o 5). eb59c9, 67 líneas

```

template<typename T, int D>
class fenwick_tree {
public:
    template<typename... P>
    fenwick_tree(int n, const P&... s)
        : mem_(n + 1, fenwick_tree<T, D - 1>(s...)) {}

    template<typename... P>
    void modify_add(int i, const P&... s) {
        for (i += 1; i < mem_.size( ); i += (i & -i)) {
            mem_[i].modify_add(s...);
        }
    }

    /*template<typename... P>
    T operator()(const P&... x) { // C++23
        return query(x..., (x + 1)...);
    }*/

    template<typename... P>
    T operator()(const P&... x) { // C++20 o menor
        return query(x..., (x + 1)...);
    }

    template<typename... P>
    T query(const P&... x) {
        static_assert(sizeof...(P) == 2 * D);
        return query(make_index_sequence<D>( ), array<int, 2 * D>(x...
            ↪ ));
    }

    template<typename... P>
    T query_until(int f, const P&... s) const {
        T res = 0;
        for (; f != 0; f -= (f & -f)) {
            res += mem_[f].query_until(s...);
        }
        return res;
    }

private:
    template<size_t... I, typename... P>
    T query(index_sequence<I...> i, const array<int, 2 * D>& indices)
        ↪ {
        T res = 0;
        for (unsigned i = 0; i < (1 << D); ++i) {
            res += (popcount(i) % 2 == D % 2 ? +1 : -1) *
                ↪ query_until(indices[bool(i & (1 << I)) * D + I]...);
        }
        return res;
    }

    vector<fenwick_tree<T, D - 1>> mem_;

```

```

};

template<typename T>
class fenwick_tree<T, 0> {
public:
    void modify_add(const T& d) {
        v_ += d;
    }

    T query_until( ) const {
        return v_;
    }

private:
    T v_ = 0;
};

```

Árbol de segmentos

Descripción: Estructura de datos para monoides $(T, \cdot : T \times T \rightarrow T, n \in T)$, permite realizar actualizaciones de elementos y calcular el producto de los elementos en un intervalo.

Uso: auto s = segment_tree(move(vector_inicial), assoc_op(0, plus()));
 int suma1 = s.query(5, 10);
 s.replace(2, rand());
 int suma2 = s.query(5, 10);

Complejidad: $\mathcal{O}(\log n)$, se asume que f es de tiempo constante. d40bd5, 68 líneas

```

template<typename T, typename F = const T&(*)(const T&, const T&)>
struct assoc_op {
    T neutro;
    F funcion;
};

```

```

template<typename OP>
struct segment_tree {
    const OP op;
    using T = decltype(OP::neutro);

    segment_tree(vector<T>&& v, OP p)
        : op(move(p)) {
        for (pisos.push_back(move(v)); pisos.back( ).size( ) > 1; ) {
            pisos.emplace_back(pisos.back( ).size( ) / 2);
            for (int i = 0, k = pisos.size( ) - 2; i < pisos[k].size( ) /
                ↪ 2; ++i) {
                pisos.back( )[i] = op.funcion(pisos[k][2 * i], pisos[k][2 *
                    ↪ i + 1]);
            }
        }
    }

    int size( ) const {
        return pisos[0].size( );
    }

    const T& operator[](int i) const {
        return pisos[0][i];
    }

    void replace(int i, T v) {
        pisos[0][i] = move(v);
        for (int p = 0; i - i % 2 + 1 != pisos[p].size( ); ++p, i /= 2)
            ↪ {
            pisos[p + 1][i / 2] = op.funcion(pisos[p][i - i % 2],
                ↪ pisos[p][i - i % 2 + 1]);
        }
    }

    T query(int ini, int fin) const {
        T res = op.neutro;
        visit(ini, fin, [&](const T& valor) {
            res = op.funcion(res, valor);
        });
        return res;
    }

    template<typename V>
    void visit(int ini, int fin, V&& vis) const {
        const T* derecha[64], **w = &derecha[0];
        for (int p = 0; ini != fin; ++p, ini /= 2, fin /= 2) {
            if (ini % 2 == 1) {
                vis(pisos[p][ini++]);
            }
            if (fin % 2 == 1) {
                *w++ = &pisos[p][--fin];
            }
        }
        while (w != &derecha[0]) {
            vis(**w);
        }
    }
};

```

```
private:
    vector<vector<T>> pisos;
};

template<typename T, typename... P> // función sólo necesaria para
↪ hld
auto make_segment_tree(vector<T>&& v, assoc_op<P...> a) {
    return segment_tree(move(v), a);
}
```

Árbol de segmentos perezoso

Descripción: Árbol de segmentos con capacidad para modificar valores de intervalos grandes y calcular consultas de intervalos.

Uso: `auto s = lazy_segment_tree(move(vector_inicial), lazy_assoc_op(`

```
    0,
    0,
    plus( ),
    [](int& valor, int cubiertos, int cambio) {
        valor += cambio * cubiertos;
        return true;
    },
    [](int cambio1, int cambio2) {
        return cambio1 + cambio2;
    }));
int suma1 = s.query(5, 10);
s.update_with(2, 8, +1);
int suma2 = s.query(5, 10);
s.override_with(0, 5, [](pair<int, int>& nodo, int cubiertos) {

    // cubiertos == 1, aplicar la modificación deseada y devolver
cualquier booleano
    // cubiertos > 1, devolver un booleano que indique si quieres
seguir descendiendo
});
```

Complejidad: $\mathcal{O}(\log n)$

eb7c87, 97 líneas

```
template<typename T, typename U, typename FQ = const T&(*)(const T&,
↪ const T&), typename FU = bool(*) (T&, int, const U&), typename FP
↪ = const U&(*)(const U&, const U&>
struct lazy_assoc_op {
    T neutro;
    U neutro_update;
    FQ funcion;
    FU funcion_update;
    FP funcion_propagar;
};
```

```
template<typename OP>
struct lazy_segment_tree {
    const OP op;
    using T = decltype(OP::neutro);
    using U = decltype(OP::neutro_update);

    lazy_segment_tree(vector<T>&& init, OP p)
    : op(move(p)), mem(init.size( ) * 2) {
        if (auto p = init.data( ); !init.empty( )) {
            construye(0, 0, size( ), p);
        }
    }

    int size( ) const {
        return mem.size( ) / 2;
    }

    T operator[](int i) const {
        return query(i, i + 1);
    }

    T query(int ini, int fin) const {
        T res = op.neutro;
        visit(0, ini, fin, 0, size( ), [&](const pair<T, U>& actual, int
↪ cubiertos) {
            res = op.funcion(res, actual.first);
            return false;
        });
        return res;
    }

    void update_with(int ini, int fin, const U& u) {
        visit(0, ini, fin, 0, size( ), [&](pair<T, U>& actual, int
↪ cubiertos) {
            actualiza(actual, cubiertos, u);
            return false;
        });
    }

    template<typename O>
    void override_with(int ini, int fin, const O& ov) {
        visit(0, ini, fin, 0, size( ), [&](pair<T, U>& actual, int
↪ cubiertos) {
            return ov(actual, cubiertos);
        });
    }
};
```

```
    });
}

template<typename V>
void visit(int ini, int fin, V&& vis) const {
    visit(0, ini, fin, 0, size( ), [&](const pair<T, U>& actual, int
↪ cubiertos) {
        vis(actual.first, cubiertos);
        return false;
    });
}
```

```
private:
    const pair<T, U>& construye(int i, int ini, int fin, T& p) {
        if (fin - ini == 1) {
            return mem[i] = { move(*p++), op.neutro_update };
        } else {
            int tam = fin - ini, mitad = ini + tam / 2, izq = i + 1, der =
↪ i + 2 * (tam / 2);
            auto t1 = construye(izq, ini, mitad, p).first, t2 =
↪ construye(der, mitad, fin, p).first;
            return mem[i] = { op.funcion(t1, t2), op.neutro_update };
        }
    }
}
```

```
template<typename V>
void visit(int i, int qi, int qf, int ini, int fin, V&& vis) const
↪ {
    if (qi < qf && (qi != ini || qf != fin || vis(mem[i], fin - ini)
↪ && fin - ini > 1)) {
        int tam = fin - ini, mitad = ini + tam / 2, izq = i + 1, der =
↪ i + 2 * (tam / 2);
        actualiza(mem[izq], tam / 2, mem[i].second);
        actualiza(mem[der], tam - tam / 2, mem[i].second);
        visit(izq, qi, min(qf, mitad), ini, mitad, vis);
        visit(der, max(qi, mitad), qf, mitad, fin, vis);
        mem[i] = { op.funcion(mem[izq].first, mem[der].first),
↪ op.neutro_update };
    }
}
```

```
void actualiza(pair<T, U>& actual, int cubiertos, const U& cambio)
↪ const {
    if (cambio != op.neutro_update &&
↪ op.funcion_update(actual.first, cubiertos, cambio)) {
        actual.second = (actual.second != op.neutro_update ?
↪ op.funcion_propagar(actual.second, cambio) : cambio);
    }
}
```

```
mutable vector<pair<T, U>> mem;
};
```

```
template<typename T, typename... P> // función sólo necesaria para
↪ hld
auto make_segment_tree(vector<T>&& v, lazy_assoc_op<P...> a) {
    return lazy_segment_tree(move(v), a);
}
```

Árbol de segmentos persistente

Descripción: Árbol de segmentos que permite consultar cualquier versión anterior del árbol después de cada modificación.

Uso: `auto s = persistent_segment_tree(move(vector_inicial), as-`
`soc_op(0, plus( ));`
`int suma1 = s.query(5, 10);`
`auto s2 = s.replace(i, rand( ));`
`int suma2 = s.query(5, 10);`

Complejidad: $\mathcal{O}(\log n)$

Requiere: Árbol de segmentos

7dcfdb, 95 líneas

```
template<typename OP>
class persistent_segment_tree {
public:
    const OP op;
    using T = decltype(OP::neutro);

    struct nodo {
        T valor;
        nodo *izq, *der;
    };

    persistent_segment_tree(vector<T>&& v, OP p)
    : mem(make_shared<deque<nodo>>( )), op(move(p)), tam(v.size( )) {
        auto ini = v.data( );
        raiz = replace(nullptr, 0, size( ), 0, size( ), [&]{
            return move(*ini++);
        });
    }

    int size( ) const {
        return tam;
    }
};
```

```

}

const nodo* root( ) const {
    return raiz;
}

const T& operator[](int i) const {
    const T* res;
    visit(i, i + 1, [&](const T& valor) {
        res = &valor;
    });
    return *res;
}

T query(int ini, int fin) const {
    T res = op.neutro;
    visit(ini, fin, [&](const T& valor) {
        res = op.funcion(res, valor);
    });
    return res;
}

persistent_segment_tree replace(int i, T v) {
    return { mem, op, size( ), replace(raiz, i, i + 1, 0, size( ),
        ↳ [&]{
            return move(v);
        }) };
}

template<typename V>
void visit(int ini, int fin, V&& vis) const {
    return visit(raiz, ini, fin, 0, size( ), vis);
}

private:
persistent_segment_tree(shared_ptr<deque<nodo>>& m, OP p, int t,
    ↳ nodo* r)
: mem(m), op(move(p)), tam(t), raiz(r) {
}

template<typename I>
nodo* replace(nodo* p, int qi, int qf, int ini, int fin, I&&
    ↳ entrada) {
    if (qi >= qf) {
        return p;
    } else if (fin - ini == 1) {
        return crea(entrada( ));
    } else {
        int mitad = ini + (fin - ini) / 2;
        auto izq = replace((p == nullptr ? nullptr : p->izq), qi,
            ↳ min(qf, mitad), ini, mitad, entrada);
        auto der = replace((p == nullptr ? nullptr : p->der), max(qi,
            ↳ mitad), qf, mitad, fin, entrada);
        return crea(op.funcion(izq->valor, der->valor), izq, der);
    }
}

template<typename V>
void visit(const nodo* p, int qi, int qf, int ini, int fin, V&
    ↳ vis) const {
    if (qi == ini && qf == fin && ini != fin) {
        vis(p->valor);
    } else if (qi < qf) {
        int mitad = ini + (fin - ini) / 2;
        visit(p->izq, qi, min(qf, mitad), ini, mitad, vis);
        visit(p->der, max(qi, mitad), qf, mitad, fin, vis);
    }
}

template<typename... P>
nodo* crea(P&&... v) {
    return &*mem->insert(mem->end( ), nodo{forward<P>(v)...});
}

shared_ptr<deque<nodo>> mem;
int tam;
nodo* raiz;
};

// < C++17 checar segment_tree

```

Árboles con estadísticas

Descripción: Implementación de árboles ordenados con índices a partir de 0 para sus nodos

Uso: statistics_set<int> s;
s.insert(3), s.insert(5), s.insert(7);
cout << s.order_of_key(5) << "\n"; // índice dada la clave de búsqueda
cout << *s.find_by_order(1) << "\n"; // iterador al nodo dado el índice

Complejidad: $\mathcal{O}(\log n)$

1ea4f5, 12 líneas

```
#include <ext/pb_ds/assoc_container.hpp>
```

```
#include <ext/pb_ds/tree_policy.hpp>
```

```
namespace gnu = __gnu_pbds;
```

```
template<typename T>
using statistics_set = gnu::tree<T, gnu::null_type, less<T>,
    ↳ gnu::rb_tree_tag, gnu::tree_order_statistics_node_update>;
```

```
template<typename T, typename V>
using statistics_map = gnu::tree<T, V, less<T>, gnu::rb_tree_tag,
    ↳ gnu::tree_order_statistics_node_update>;
```

// para multiset y multimap, hay un "hack" que usa less_equal, pero
↳ .lower_bound ya no funciona, aunque .upper_bound sí

Geométricos (7)

Cerco convexo

Descripción:

Devuelve un vector de los puntos del cerco convexo en sentido antihorario. Los puntos en el borde del cerco entre dos puntos no se consideran parte del cerco.

Uso: sort(puntos.begin(), puntos.end());
auto cerco = cerco_convexo(puntos.begin(), puntos.end());

Complejidad: $\mathcal{O}(n \log n)$

3832e6, 34 líneas

```

struct punto {
    double x, y; // si no necesitan doubles, pasarlos a int porque
    ↳ es más rápido
    bool operator<(const punto& p) const {
        return pair(x, y) < pair(p.x, p.y);
    }
};

auto producto_cruz(const auto& a, const auto& b, const auto& c) {
    return (b.x - a.x) * (c.y - a.y) - (b.y - a.y) * (c.x - a.x);
}

template<typename RI1, typename RI2>
auto cerco_parcial(RI1 ai, RI1 af, RI2 bi) {
    auto bw = bi;
    for (; ai != af; *bw++ = *ai++) {
        while (bw - bi >= 2 && producto_cruz(*(bw - 2), *(bw - 1), *ai)
            ↳ <= 0) { // < 0 para permitir empates en línea recta
            --bw;
        }
    }
    return bw;
}

template<typename RI>
auto cerco_convexo(RI ai, RI af) {
    if (af - ai <= 2) {
        return vector<iter_value_t<RI>>(ai, af);
    } else {
        vector<iter_value_t<RI>> res(2 * (af - ai));
        auto it1 = cerco_parcial(ai, af, res.begin( )) - 1;
        auto it2 = cerco_parcial(reverse_iterator(af),
            ↳ reverse_iterator(ai), it1) - 1;
        res.resize(it2 - res.begin( ));
        return res;
    }
}

```

Distancia entre dos puntos

Descripción: Calcula la distancia euclidiana entre dos puntos en un plano.

Uso: double dis = distancia<int>({ 0, 0 }, { 1, 1 });

31a15d, 6 líneas

```

template<typename T>
double distancia(const pair<T, T>& a, const pair<T, T>& b) {
    auto dx = a.first - b.first;
    auto dy = a.second - b.second;
    return sqrt(dx * dx + dy * dy);
}

```

Gráficas (8)

Algoritmo de Bellman-Ford

Descripción: El algoritmo de Bellman-Ford encuentra los caminos más cortos desde un vértice inicial *s* hacia todos los demás vértices de una gráfica dirigida con costos negativos (pero sin ciclos negativos).

Complejidad: $\mathcal{O}(nm)$

99050e, 31 líneas

```

struct tupla {
    int origen, destino, costo;
}

```

```

};

int main( ) {
    int n, m;
    cin >> n >> m;

    vector<vector<tuple>> adyacencia(n);
    for (int i = 0; i < m; ++i) {
        int x, y, c;
        cin >> x >> y >> c;
        adyacencia[x].push_back({ x, y, c });    // sólo en un sentido:
        ↪ deben ser arcos
    }

    vector<int> distancias(n, 1e9);
    distancias[0] = 0;
    for (int k = 0; k < n - 1; ++k) {
        for (int i = 0; i < n; ++i) {
            if (distancias[i] != 1e9) {
                for (tuple t : adyacencia[i]) {
                    distancias[t.destino] = min(distancias[t.destino],
                    ↪ distancias[i] + t.costo);
                }
            }
        }
    }

    for (int i = 0; i < n; ++i) {
        cout << i << ": " << distancias[i] << "\n";
    }
}

```

Algoritmo de Edmonds-Karp

Descripción: Algoritmo de flujo máximo de una gráfica sin costos de n vértices y m aristas.

Uso: edmonds_karp<int> f(n, s, t);
 f.agrega_arco(i, j, c);
 int flujo = f.flujo_maximo();
 int t = f.flujo_arco(i, j);

Complejidad: $\mathcal{O}(nm^2)$ d3c6b8, 65 líneas

```

template<typename C>
struct edmonds_karp {
    int fuente, sumidero;
    vector<vector<int>> adj;
    vector<vector<C>> cap;

    edmonds_karp(int n, int f, int s)
    : fuente(f), sumidero(s), adj(n), cap(n, vector<C>(n)) {}

    void agrega_arco(int i, int j, C c) {
        if (i != j && c != 0) {
            if (cap[i][j] == 0 && cap[j][i] == 0) {
                adj[i].push_back(j);
                adj[j].push_back(i);
            }
            cap[i][j] += c;
        }
    }

    C flujo_arco(int i, int j) {
        return cap[j][i];    // sí, así
    }

    C flujo_maximo( ) {
        C res = 0;
        C res = 0;
        for (;;) {
            auto [delta, camino] = aumenta( );
            if (delta == 0) {
                return res;
            }
            res += delta;
            for (auto [i, j] : camino) {
                cap[i][j] -= delta;
                cap[j][i] += delta;
            }
        }
    }

private:
    pair<C, vector<pair<int, int>>> aumenta( ) {
        vector<int> anterior(adj.size( ), -1);
        vector<pair<int, int>> camino;
        anterior[fuente] = fuente;
        for (deque<int> cola = { fuente }; !cola.empty( );
        ↪ cola.pop_front( )) {
            int i = cola.front( );
            if (i == sumidero) {
                C cuello = numeric_limits<C>::max( );
                vector<pair<int, int>> camino;
                for (int i = sumidero; i != fuente; i = anterior[i]) {
                    camino.emplace_back(anterior[i], i);
                    cuello = min(cuello, cap[anterior[i]][i]);
                }
            }
        }
    }
}

```

```

    }
    return { cuello, camino };
}
for (int j : adj[i]) {
    if (anterior[j] == -1 && cap[i][j] > 0) {
        cola.push_back(j);
        anterior[j] = i;
    }
}
}
return { 0, { } };
}
};

```

Algoritmo de Edmonds-Karp con costos usando Bellman-Ford

Descripción: Algoritmo de flujo máximo de costo mínimo de una gráfica de n vértices y m aristas.

Uso: edmonds_karp_bf<int, int> f(n, s, t);
 f.agrega_arco(i, j, c);
 auto [flujo, costo] = f.flujo_maximo();
 int t = f.flujo_arco(i, j);

Complejidad: $\mathcal{O}(Fnm)$ e5ede8, 69 líneas

```

template<typename C, typename D>
struct edmonds_karp_bf {
    int fuente, sumidero;
    vector<vector<int>> adj;
    vector<vector<C>> cap;
    vector<vector<D>> costo;

    edmonds_karp_bf(int n, int f, int s)
    : fuente(f), sumidero(s), adj(n), cap(n, vector<C>(n)), costo(n,
    ↪ vector<C>(n)) {}

    void agrega_arco(int i, int j, C c, D d) {    // no se admiten arcos
        ↪ paralelos
        if (i != j && c != 0) {
            adj[i].push_back(j);
            adj[j].push_back(i);
            cap[i][j] = c;
            costo[i][j] = d;
            costo[j][i] = -d;
        }
    }

    C flujo_arco(int i, int j) {
        return cap[j][i];    // sí, así
    }

    pair<C, D> flujo_maximo( ) {
        C flujo_total = 0; D costo_total = 0;
        for (;;) {
            auto [delta, camino] = aumenta( );
            if (delta == 0) {
                return { flujo_total, costo_total };
            }
            flujo_total += delta;
            for (auto [i, j] : camino) {
                costo_total += delta * costo[i][j];
                cap[i][j] -= delta;
                cap[j][i] += delta;
            }
        }
    }

private:
    pair<C, vector<pair<int, int>>> aumenta( ) {
        vector<int> anterior(adj.size( ), -1);
        vector<D> min(adj.size( ), numeric_limits<D>::max( ) / 2);
        min[fuente] = 0;    // la división está por riesgo de overflow
        for (int k = 0; k < adj.size( ) - 1; ++k) {
            for (int i = 0; i < adj.size( ); ++i) {
                for (int j : adj[i]) {
                    if (cap[i][j] > 0 && min[j] > min[i] + costo[i][j]) {
                        min[j] = min[i] + costo[i][j];
                        anterior[j] = i;
                    }
                }
            }
        }
        if (anterior[sumidero] == -1) {
            return { 0, { } };
        }

        C cuello = numeric_limits<C>::max( );
        vector<pair<int, int>> camino;
        for (int i = sumidero; i != fuente; i = anterior[i]) {
            camino.emplace_back(anterior[i], i);
            cuello = min(cuello, cap[anterior[i]][i]);
        }
    }
}

```

```

    }
    return { cuello, camino };
}
};

```

Algoritmo de Edmonds-Karp con costos usando el algoritmo de Johnson

Descripción: Algoritmo de flujo máximo de costo mínimo de una gráfica de n vértices y m aristas.

Uso: edmonds_karp_bf<int, int> f(n, s, t);
 f.agrega_arco(i, j, c);
 auto [flujo, costo] = f.flujo_maximo();
 int t = f.flujo_arco(i, j);

Complejidad: $\mathcal{O}(Fm \log m)$

90f354, 79 líneas

```

template<typename C, typename D>
struct edmonds_karp_j {
    int fuente, sumidero;
    vector<vector<int>> adj;
    vector<vector<C>> cap;
    vector<vector<D>> costo;

    edmonds_karp_j(int n, int f, int s)
    : fuente(f), sumidero(s), adj(n), cap(n, vector<C>(n)), costo(n,
      vector<D>(n)) {

    void agrega_arco(int i, int j, C c, D d) { // no se admiten arcos
      // paralelos ni costos negativos
      if (i != j && c != 0) {
          adj[i].push_back(j);
          adj[j].push_back(i);
          cap[i][j] = c;
          costo[i][j] = d;
          costo[j][i] = -d;
      }
    }

    C flujo_arco(int i, int j) {
        return cap[j][i]; // si, así
    }

    pair<C, D> flujo_maximo( ) {
        C flujo_total = 0; D costo_total = 0;
        vector<D> potencial(adj.size( ), 0);
        for (;;) {
            auto [delta, camino] = aumenta(potencial);
            if (delta == 0) {
                return { flujo_total, costo_total };
            }
            flujo_total += delta;
            for (auto [i, j] : camino) {
                costo_total += delta * costo[i][j];
                cap[i][j] -= delta;
                cap[j][i] += delta;
            }
        }
    }

private:
    pair<C, vector<pair<int, int>>> aumenta(vector<int>& potencial) {
        vector<int> anterior(adj.size( ), -1);
        priority_queue<tuple<D, int, int>, vector<tuple<D, int, int>>,
          greater<>> cp;
        vector<D> min(adj.size( ), numeric_limits<D>::max( ));
        cp.emplace(0, fuente, fuente);
        do {
            auto [d, i, a] = cp.top( );
            cp.pop( );
            if (anterior[i] == -1) {
                anterior[i] = a, min[i] = d;
                for (int j : adj[i]) {
                    if (cap[i][j] > 0) {
                        cp.emplace(d + costo[i][j] + potencial[i] -
                          potencial[j], j, i);
                    }
                }
            }
        } while (!cp.empty( ));
        if (anterior[sumidero] == -1) {
            return { 0, { } };
        }

        C cuello = numeric_limits<C>::max( );
        vector<pair<int, int>> camino;
        for (int i = sumidero; i != fuente; i = anterior[i]) {
            camino.emplace_back(anterior[i], i);
            cuello = min(cuello, cap[anterior[i]][i]);
        }
        for (int i = 0; i < adj.size( ); ++i) {
            if (min[i] != numeric_limits<D>::max( )) {
                min[i] += potencial[i] - potencial[fuente];
            }
        }
    }
}

```

```

    }
    swap(potencial, min);
    return { cuello, camino };
}
};

```

Algoritmo de Floyd

Descripción: Calcula el camino más corto entre cualquier pareja de vértices en un grafo dirigido.

Complejidad: $\mathcal{O}(n^3)$

f7e22a, 9 líneas

```

// adyacencia[i][j] es la matriz original
// la siguiente implementación la modifica
for (int k = 0; k < n; ++k) {
    for (int i = 0; i < n; ++i) {
        for (int j = 0; j < n; ++j) {
            adyacencia[i][j] = min(adyacencia[i][j], adyacencia[i][k] +
              adyacencia[k][j]);
        }
    }
}

```

Algoritmo de Hopcroft-Karp

Descripción: Algoritmo rápido de acoplamiento bipartito. Devuelve el tamaño del acoplamiento. *match_b[j]* será el vértice del lado izquierdo que quedó emparejado con el vértice j del lado derecho, o -1 si no está emparejado.

Uso: bipartito_hopcroft bp(|A|, |B|);
 bp.agrega_arco(i, j); // por cada arista ($i \in A, j \in B$)
 int tam = bp.acoplamiento_maximo();
 if (int i = bp.match_b[j]; i != -1) {
 // i está emparejado con j
 }

Complejidad: $\mathcal{O}(\sqrt{nm})$

ea4269, 63 líneas

```

struct bipartito_hopcroft {
    vector<vector<int>> adj;
    vector<int> match_b;

    bipartito_hopcroft(int n, int m)
    : adj(n), match_b(m, -1) {

    void agrega_arco(int i, int j) {
        adj[i].push_back(j);
    }

    int acoplamiento_maximo( ) {
        for (;;) {
            vector<int> capas = calcula_capas( );
            bool mejora = false, visto_a[adj.size( )] = { };
            for (int i = 0; i < adj.size( ); ++i) {
                mejora |= (capas[i] == 0 && aumenta(i, visto_a, capas));
            }
            if (!mejora) {
                return match_b.size( ) - count(match_b.begin( ),
                  match_b.end( ), -1);
            }
        }
    }

private:
    vector<int> calcula_capas( ) {
        vector<int> capas(adj.size( ), 0);
        deque<int> cola(adj.size( ));
        iota(cola.begin( ), cola.end( ), 0);
        for (int j = 0; j < match_b.size( ); ++j) {
            int i = match_b[j];
            if (i != -1) {
                cola[i] = capas[i] = -1;
            }
        }

        cola.erase(remove(cola.begin( ), cola.end( ), -1), cola.end( ));
        for (; !cola.empty( ); cola.pop_front( )) {
            int i = cola.front( );
            for (int j : adj[i]) {
                if (match_b[j] != -1 && capas[match_b[j]] == -1) {
                    capas[match_b[j]] = capas[i] + 1;
                    cola.push_back(match_b[j]);
                }
            }
        }
        return capas;
    }

    bool aumenta(int i, bool visto_a[], const vector<int>& capas) {
        if (!visto_a[i]) {
            visto_a[i] = true;

```

```

    for (int j : adj[i]) {
        if (match_b[j] == -1 || capas[i] + 1 == capas[match_b[j]] &&
            ↪ aumenta(match_b[j], visto_a, capas)) {
            match_b[j] = i;
            return true;
        }
    }
}
return false;
};

```

Algoritmo de Kruskal

Descripción: Dado un grafo no dirigido y ponderado, se busca encontrar un árbol abarcador con el menor costo posible.

Complejidad: $\mathcal{O}(m \log n)$

Requiere: Unión-pertenencia

e6da5a, 27 líneas

```

struct arista {
    int x, y, costo;
};

int main( ) {
    int v, a;
    cin >> v >> a;

    vector<arista> aristas;
    for (int i = 0; i < a; ++i) {
        int x, y, costo;
        cin >> x >> y;
        aristas.push_back({ x, y, costo });
    }

    sort(aristas.begin( ), aristas.end( ), [](arista a, arista b) {
        return a.costo < b.costo;
    });

    union_find uf(v);

    for (int i = 0; i < aristas.size( ); ++i) {
        if (!uf.connected(aristas[i].x, aristas[i].y)) {
            uf.join(aristas[i].x, aristas[i].y);
        }
    }
}

```

Algoritmo de Kuhn

Descripción: Algoritmo simple de acoplamiento bipartito. Devuelve el tamaño del acoplamiento. *match_b[j]* será el vértice del lado izquierdo que quedó emparejado con el vértice *j* del lado derecho, o -1 si no está emparejado.

Uso: bipartito_kuhn bp(|A|, |B|);
 bp.agrega_arco(i, j); // por cada arista ($i \in A$, $j \in B$)
 int tam = bp.acoplamiento_maximo();
 if (int i = bp.match_b[j]; i != -1) {
 // i está emparejado con j
 }

Complejidad: $\mathcal{O}(nm)$

2702f9, 36 líneas

```

struct bipartito_kuhn {
    vector<vector<int>> adj;
    vector<int> match_b;

    bipartito_kuhn(int n, int m)
    : adj(n), match_b(m, -1) {}

    void agrega_arco(int i, int j) {
        adj[i].push_back(j);
    }

    int acoplamiento_maximo( ) {
        int res = 0;
        fill(match_b.begin( ), match_b.end( ), -1);
        for (int i = 0; i < adj.size( ); ++i) {
            bool visto_a[adj.size( )] = { };
            res += aumenta(i, visto_a);
        }
        return res;
    }

private:
    bool aumenta(int i, bool visto_a[]) {
        if (!visto_a[i]) {
            visto_a[i] = true;
            for (int j : adj[i]) {
                if (match_b[j] == -1 || aumenta(match_b[j], visto_a)) {
                    match_b[j] = i;
                    return true;
                }
            }
        }
    }
}

```

```

    }
    return false;
}
};

```

Algoritmo de push-relabel

Descripción: Algoritmo rápido de flujo máximo de una gráfica sin costos de n vértices y m aristas.

Uso: push_relabel<int> f(n, s, t);
 f.agrega_arco(i, j, c);
 int flujo = f.flujo_maximo();
 int t = f.flujo_arco(i, j);

Complejidad: $\mathcal{O}(n^2\sqrt{m})$

37769e, 83 líneas

```

template<typename C>
struct push_relabel {
    int fuente, sumidero;
    vector<vector<int>> adj;
    vector<vector<C>> cap;
    vector<C> exceso;
    vector<int> altura;

    push_relabel(int n, int f, int s)
    : fuente(f), sumidero(s), adj(n), cap(n, vector<C>(n, 0)),
      exceso(n, 0), altura(n, 0) {
        exceso[fuente] = numeric_limits<C>::max( );
        altura[fuente] = n;
    }

    void agrega_arco(int i, int j, C c) {
        if (i != j && c != 0) {
            if (cap[i][j] == 0 && cap[j][i] == 0) {
                adj[i].push_back(j);
                adj[j].push_back(i);
            }
            cap[i][j] += c;
        }
    }

    C flujo_arco(int i, int j) {
        return cap[j][i]; // sí, así
    }

    C flujo_maximo( ) {
        vector<int> por_altura[2 * adj.size( ) + 1];
        for (int j : adj[fuente]) {
            push(fuente, j, por_altura);
        }

        int max_altura = 0;
        while (busca_alto(por_altura, max_altura)) {
            int i = por_altura[max_altura].back( );
            por_altura[max_altura].pop_back( );
            for (int x = 0; x < adj[i].size( ) && exceso[i] != 0; ++x) {
                if (altura[i] - 1 == altura[adj[i][x]]) {
                    push(i, adj[i][x], por_altura);
                }
            }
            if (exceso[i] != 0) {
                relabel(i, por_altura, max_altura);
            }
        }
        return exceso[sumidero];
    }

private:
    bool busca_alto(vector<int> por_altura[], int& max_altura) {
        while (max_altura >= 0 && por_altura[max_altura].empty( )) {
            max_altura -= 1;
        }
        return max_altura >= 0;
    }

    void push(int i, int j, vector<int> por_altura[]) {
        C flujo = min(exceso[i], cap[i][j]);
        if (flujo != 0) {
            if (exceso[j] == 0 && j != fuente && j != sumidero) {
                por_altura[altura[j]].push_back(j);
            }
            exceso[i] -= flujo;
            exceso[j] += flujo;
            cap[i][j] -= flujo;
            cap[j][i] += flujo;
        }
    }

    void relabel(int i, vector<int> por_altura[], int& max_altura) {
        int altura_min = 1e9;
        for (int j : adj[i]) {
            if (cap[i][j] != 0) {
                altura_min = min(altura_min, altura[j]);
            }
        }
    }
}

```



```

    }
    altura[i] = max_altura = altura_min + 1;
    por_altura[altura[i]].push_back(i);
}
};

```

Algoritmos de Dijkstra y Prim

Descripción: El algoritmo de Dijkstra encuentra los caminos más cortos desde un vértice inicial s hacia todos los demás vértices del grafo con costos no negativos. El árbol abarcador de costo mínimo, que se obtiene usando el algoritmo de Prim, es un conjunto de aristas que conecta todos los vértices con el menor costo total posible.

Complejidad: $\mathcal{O}(m \log n)$

32131c, 41 líneas

```

struct tupla {
    int origen, destino, costo;
};

bool operator<(tupla a, tupla b) {
    return a.costo > b.costo;
}

int main( ) {
    int n, m;
    cin >> n >> m;

    vector<tupla> adyacencia[n];
    for (int i = 0; i < m; ++i) {
        int x, y, c;
        cin >> x >> y >> c;
        adyacencia[x].push_back({ x, y, c });
        adyacencia[y].push_back({ y, x, c });
    }

    priority_queue<tupla> cp;
    cp.push({0, 0, 0});
    int costo[n], previo[n];
    fill(&costo[0], &costo[n], -1);
    do {
        tupla t = cp.top();
        cp.pop();
        if (costo[t.destino] == -1) {
            costo[t.destino] = t.costo;
            previo[t.destino] = t.origen;
            for (tupla vecino : adyacencia[t.destino]) {
                vecino.costo += t.costo; // quitar para Prim
                cp.push(vecino);
            }
        }
    } while (!cp.empty());

    for (int i = 0; i < n; ++i) {
        cout << i << ": " << costo[i] << " " << previo[i] << "\n";
    }
}

```

Circuito euleriano

Descripción: Un circuito euleriano es un camino que recorre cada arista de un grafo exactamente una vez, y el camino termina en el vértice *inicial*.

Complejidad: $\mathcal{O}(n + m)$

467b64, 17 líneas

```

vector<int> hierholzer(int inicial, vector<set<pair<int, int>>>&
↪ adyacencia) {
    vector<int> pila = { inicial }, res;
    do {
        int actual = pila.back();
        if (!adyacencia[actual].empty()) {
            auto [origen, destino] = *adyacencia[actual].begin();
            adyacencia[actual].erase(adyacencia[actual].begin());
            adyacencia[destino].erase(pair(destino, actual));
            pila.push_back(destino);
        } else {
            res.push_back(actual);
            pila.pop_back();
        }
    } while (!pila.empty());
    reverse(res.begin(), res.end());
    return res;
}

```

Puentes de una gráfica

Descripción: Dado un grafo no dirigido. Un puente se define como una arista que, al ser removida, desconecta el grafo (o, más precisamente, aumenta el número de componentes conexos en el grafo). Devuelve todos los puentes en el grafo dado.

Complejidad: $\mathcal{O}(n + m)$

140d10, 31 líneas

```

void dfs(int actual, int anterior, const vector<int> adyacencia[],
↪ vector<bool>& visitado, int& id, vector<int>& tin, vector<int>& res) {
    low[actual] = true, tin[actual] = low[actual] = id++;
    bool anterior_omitido = false;
    for (int vecino : adyacencia[actual]) {
        if (vecino == anterior && !anterior_omitido) {
            anterior_omitido = true;
            continue;
        }
        if (visitado[vecino]) {
            low[actual] = min(low[actual], tin[vecino]);
        } else {
            dfs(vecino, actual, adyacencia, visitado, id, tin, low, res);
            low[actual] = min(low[actual], low[vecino]);
            if (low[vecino] > tin[actual]) {
                res.emplace_back(min(actual, vecino), max(actual, vecino));
            }
        }
    }
}

vector<pair<int, int>> calcula_puentes(const vector<int>
↪ adyacencia[], int n) {
    vector<bool> visitado(n); int id;
    vector<int> tin(n, -1), low(n, -1);
    vector<pair<int, int>> res;
    for (int i = 0; i < n; ++i) {
        if (!visitado[i]) {
            dfs(i, -1, adyacencia, visitado, id, tin, low, res);
        }
    }
    return res;
}

```

Unión-pertenencia

Descripción: Dado un grafo no dirigido, procesa la adición de aristas, verificar si dos vértices están en el mismo componente conexo y obtener el tamaño del componente de un elemento.

Complejidad: $\mathcal{O}(\alpha(n))$ (amortizado).

f11bf1, 30 líneas

```

struct union_find {
    vector<int> data;

    union_find(int n) : data(n, -1) {}

    int leader(int i) {
        return (data[i] < 0 ? i : data[i] = leader(data[i]));
    }

    int size(int i) {
        return -data[leader(i)];
    }

    bool connected(int i, int j) {
        return leader(i) == leader(j);
    }

    void join(int i, int j) {
        int ri = leader(i), rj = leader(j);
        if (ri != rj) {
            if (-data[ri] < -data[rj]) {
                swap(ri, rj);
            }
            data[ri] += data[rj];
            data[rj] = ri;
        }
    }
};

```

Lectura y escritura (9)

Construcción de cadenas

Descripción: Ejemplo de concatenación de valores de diferentes tipos de datos en una sola cadena.

3b1190, 11 líneas

```

int main( ) {
    int a = 57;
    char c = '@';
    double f = 3.14;

    ostream bufer;
    bufer << a << " " << c << " " << f;
    string cadena = bufer.str();

    cout << cadena;
}

```

```
}
```

Tokenización de líneas

Descripción: Ejemplo de tokenización de líneas de texto, donde cada línea leída se divide en palabras que se imprimen individualmente.

f9cb58, 11 líneas

```
int main( ) {
    string linea;
    while (getline(cin, linea)) {
        istream extractor(linea);
        string palabra;
        while (extractor >> palabra) {
            cout << palabra << " ";
        }
        cout << "\n";
    }
}
```